模块2：命令行《测试计划》

1. 引言

1.1 编写目的

本测试计划用于指导 `pyinstrument` 项目模块 2“命令行（CLI）”部分的测试设计、测试执行与结果判定。测试工作结合大模型驱动的软件测试工作流完成，覆盖《测试计划》撰写、测试用例生成、测试代码执行及《测试报告》输出的全过程。

1.2 测试目标

本次测试目标如下：

1. 验证 CLI 模块是否能正确处理脚本执行、模块执行、字符串执行、session 读取等入口模式。
2. 验证参数解析、参数冲突检测、参数传递与渲染器选择逻辑是否正确。
3. 验证 session 文件的保存、加载、历史报告管理是否正确。
4. 验证 `Profiler`、`Session`、`util`、`appdirs`、`keypath` 等支撑模块在 CLI 场景下的行为是否正确。
5. 通过黑盒和白盒结合的方式，达到不低于 95% 的覆盖要求。

2. 测试范围

2.1 源码范围

- `pyinstrument/__main__.py`
- `pyinstrument/profiler.py`
- `pyinstrument/session.py`
- `pyinstrument/util.py`
- `pyinstrument/vendor/appdirs.py`
- `pyinstrument/vendor/keypath.py`

2.2 测试代码范围

原测试代码：

- test_cmdline.py
- test_cmdline_main.py
- util.py

新增测试代码：

- test_cli_module_white_box.py

3. 测试环境

- 操作系统：Windows
- Python 版本：3.13.9
- 测试框架：`pytest`
- 工作目录：`D:\LHY\SoftWare_Test`

- 软件开发 Agent：Codex

4. 测试策略

本次测试采用黑盒测试与白盒测试相结合的策略。

4.1 黑盒测试策略

黑盒测试以需求和用户视角为基础，不关心内部实现，重点验证输入与输出行为是否符合预期。

采用的方法如下：

4.1.1 等价类划分

对 CLI 输入进行分类：

- 合法入口类：
  - 脚本执行
  - `-m` 模块执行
  - `-c` 字符串执行
  - `--load` 读取 session
  - `--load-prev` 读取历史报告
- 非法组合类：
  - 多个入口模式同时指定
  - 非法 renderer 名称
  - 缺失历史 session 标识符

4.1.2 边界值分析

对以下关键边界进行设计：

- 输出文件后缀：
  - `.txt`
  - `.html`
  - `.json`
  - `.pyisession`
  - `.pstats`
- 历史报告数量边界：
  - 10 个
  - 11 个
  - 12 个
- `target_description` 边界：
  - 合法占位符 `{args}`
  - 空占位符 `{}`
  - 未知占位符 `{foo}`

4.1.3 决策表法

针对多条件组合设计用例，例如：

- `hide` 与 `hide_regex` 是否互斥
- `show`、`show_regex`、`show_all` 是否冲突
- 是否指定 `outfile`
- 是否指定 `renderer`
- 不同组合下 renderer 的创建结果

4.2 白盒测试策略

白盒测试以源码逻辑为基础，重点检查内部语句、分支、路径和数据流。

4.2.1 代码覆盖

覆盖以下关键逻辑：

- `main()` 主流程
- 参数冲突分支
- 渲染器选择与异常处理
- session 保存、加载与裁剪逻辑
- `Profiler.start/stop/reset`
- `Session.combine/resample/shorten_path`

4.2.2 基本路径覆盖

确保至少执行以下关键路径：

- 正常 CLI 执行路径
- session 读取路径
- renderer 创建成功路径
- renderer 创建失败路径
- profiler 正常停止路径
- profiler 异常停止路径

4.2.3 数据流分析

重点分析以下数据流：

- `argv -> options -> main()`
- `render_options -> keypath -> renderer`
- `session -> save -> load -> render`
- `frame_records -> combine/resample -> output`

5. 测试代码来源与作用

5.1 原测试代码作用

test_cmdline.py

这部分原测试主要承担黑盒测试职责，验证：

- 用户从命令行调用 CLI 时的真实行为

- 子进程执行和参数透传
- session 保存与再次加载
- 输出格式与退出码

test_cmdline_main.py

这部分原测试主要验证：

- renderer 参数是否能被 `main()` 正确解析
- 普通参数、JSON 参数、dotted keypath 参数是否能注入 renderer

5.2 新增测试代码作用

test_cli_module_white_box.py

新增测试代码主要补齐原测试中的白盒空白，包括：

- `compute_render_options()` 冲突与正常分支
- `get_renderer_class()` 的后缀边界与异常路径
- `create_renderer()` 的错误处理
- `report_dir()` 与历史 session 管理
- `Profiler` 生命周期与异步采样逻辑
- `Session` 内部数据处理逻辑
- `util/appdirs/keypath` 的内部边界

6. 测试用例设计

6.1 黑盒测试用例

用例编号	测试名称	方法	输入	预期结果
BB-01	脚本入口执行	等价类	`pyinstrument script.py`	成功输出 profile 结果
BB-02	模块入口执行	等价类	`pyinstrument -m module`	成功执行模块并输出
BB-03	字符串入口执行	等价类	`pyinstrument -c "program"`	程序执行成功
BB-04	session 加载	等价类	`--load=session.pyisession`	正确读取并渲染
BB-05	冲突入口	决策表	`--load file` 与脚本参数同时给出	报错退出
BB-06	show 组合冲突	决策表	`show` 与 `show_regex` 同时给出	抛出异常
BB-07	renderer 后缀边界	边界值	`.txt/.html/.json/.pyisession/.pstats`	选择正确 renderer

用例编号	测试名称	方法	输入	预期结果
BB-08	历史报告数量边界	边界值	12 个历史 session	自动裁剪旧报告
BB-09	目标描述未知占位符	边界值	`{foo}`	报错退出
BB-10	目标描述空占位符	边界值	`{}`	报错退出

6.2 白盒测试用例

用例编号	测试名称	方法	目标	预期结果
WB-01	render options 正常路径	代码覆盖	`compute_render_options()`	正确构造参数字典
WB-02	render options 冲突路径	分支覆盖	`compute_render_options()`	抛出 `OptionsParseError`
WB-03	renderer 选择边界	基本路径	`get_renderer_class()`	合法值通过，非法值报错
WB-04	renderer 创建异常	基本路径	`create_renderer()`	非法组合抛错
WB-05	历史报告加载路径	数据流分析	`save/load_report_from_temp_storage()`	session 正确保存和读取
WB-06	profiler 生命周期	基本路径	`Profiler.start/stop/reset()`	正常记录并可复位
WB-07	profiler 异步分支	分支覆盖	`_sampler_saw_call_stack()`	async 三类分支均正确
WB-08	session 合并与重采样	数据流分析	`Session.combine/resample()`	合并和采样结果正确
WB-09	path 缩短逻辑	分支覆盖	`Session.shorten_path()`	正常路径与异常路径均正确
WB-10	util/vendor 工具逻辑	代码覆盖	`util/appdirs/keypath`	返回值符合预期

7. 自动化执行流程

1. 阅读作业需求，明确模块 2 的测试范围。
2. 阅读源码与原测试代码，识别已有黑盒覆盖点。

3. 定位白盒未覆盖的函数和分支。
4. 生成新增测试文件 `test_cli_module_white_box.py`。
5. 执行自动化测试。
6. 统计结果并分析黑盒与白盒覆盖情况。
7. 形成测试报告。

8. 通过准则

本次测试通过的标准如下：

1. 新增自动化测试全部通过。
2. 黑盒测试体现边界值、等价类、决策表三类方法。
3. 白盒测试体现代码覆盖、基本路径覆盖、数据流分析三类方法。
4. 核心代码覆盖率达到课程要求。
5. 对未来扩展场景给出测试调整方案。